

Graduate Short Essay Answers

1.

Semi-invasive attack is a limited impact attack on target hardware/software where an attacker interacts with a system in a way that reads or perturbs state without permanently altering hardware or fully taking over the platform. For example: reading memory via debug interfaces, dumping firmware via an exposed diagnostic port, or running unprivileged code that compromises data. This type of attack typically leaves the system operational and often traceable.

Invasive attack is where an attacker physically tampers with or fully compromises the target to change hardware/firmware, install persistent implants, alter system logic, or even erase and/or modify evidence. For example: opening the chassis to reflash firmware, soldering implants, or replacing chips. It basically provides higher risk, access, and harder to detect or even repair.

In a nutshell, **Invasive attack** requires physical access and specialized tools along with skills, whereas, **Semi-invasive** may be done with limited physical access or privileged logical access. To simply put in direct and simple terms, we can look at this way that the word “Invasiveness” correlates with permanence, risk of damage, and legal exposure.

2.

In an organization, an insider already have some legitimate level of access or privileged knowledge. Of course capabilities depend on role, access level and intent, the overview of what an insider can do to harm is briefly listed below.

1. Insiders can read, copy, or even move sensitive files such as customer data, IP, credentials and such using legitimate channels.
2. They can misuse elevated accounts such as admin/DBA to bypass controls, change permission levels, or even enable backdoors.
3. An insider can alter firewall rules, logging, retention, or audit settings to hide activity or open access.
4. Use of internal knowledge to deceive coworkers and/or vendors easily, can be done as well.
5. Deletion and/or tampering data, corrupting backups, or disabling monitoring can be done according the permission/access level.

Apart from above points, tampering with logs and/or timestamps are other serious damage that can be done to an organization by their insiders. Even if an insider has low privilege, they can still cause harm as being a part of their institution, they may at least and most probably have read only access. And revealing any (low to high) severity form of data is a loss nevertheless. Thus, it all comes at last to proper security implementation along with the most important factor – Trust, Loyalty, and Honesty.

3.

Design mistakes are one of the main and severe source where vulnerabilities arises from. Design errors create insecure defaults, insufficient separation of duties, ambiguous trust boundaries, or insecure assumptions about inputs and/or actors. To add on it, early design flaws propagate throughout the system and are costly to fix later. Some of the common category of design mistakes are missing authentication/authorization checks, insecure default credentials, excessive privilege granted to components, lack of input validation weak cryptography choices, poor error/exception handling that leaks information, and many more.

To demonstrate a practical example, let us look at the following code (partial) of my personal project.

```

export async function createGRNDataOfTenant(
  body: GRNAttributes
): Promise<ResponseBodyType<GRNAttributes>> {
  const cookieStore = await cookies();
  const response: ResponseBodyType<GRNAttributes> = {
    success: false,
    message: ERR_INTERNAL_SERVER,
  };
  const decodedToken = await decodeJwtForUserAuthentication(
    cookieStore.get(KeysForCookies.token)?.value || ""
  );
  if (!decodedToken || isErrorType(decodedToken)) {
    response.success = false;
    response.message = { ...ERR_AUTHORIZATION };
    return response;
  }
}

```

In the above code, we can clearly notice that there is no permission check for the GRN's *create* action. The code just validates JWT token of the cookie header and begins creating the data. In this above example, the code simply assumes that if a user has logged in and is valid (with proper JWT), the user can proceed creating the model (GRN) data. The code doesn't check the fact that whether or not a model (Contact, Quotation, LPO, GRN in this example) is creatable AND whether or not the user has permission to create it. This is a straight and severe design mistake and should be handled immediately. Imagine such severe flaw for critical actions like update and delete, it would pose even a greater risk.

However, with a little bit of design changes and its proper implementation as demonstrated in below code, the mistake can be corrected and such severe issue can be resolved.

```

export async function createGRNDataOfTenant(
  body: GRNAttributes
): Promise<ResponseBodyType<GRNAttributes>> {
  const cookieStore = await cookies();
  const response: ResponseBodyType<GRNAttributes> = {
    success: false,
    message: ERR_INTERNAL_SERVER,
  };
  // begin permission check
  const modelData = await readUserMenu({
    key: "tenant_grn",
  });
  if (isErrorType(modelData.message)) {
    response.success = false;
    response.message = modelData.message;
    return response;
  }
  if (
    You, 7 days ago • grnDetail in progress. after that, GRN nee
    !(modelData.message[0] || {}).creatable ||
    !(modelData.message[0] || {}).canCreate
  ) {
    response.success = false;
    response.message = { ...ERR_AUTHORIZATION };
    return response;
  }
  // end permission
  const decodedToken = await decodeJwtForUserAuthentication(
    cookieStore.get(KeysForCookies.token)?.value || ""
  );
  if (!decodedToken || isErrorType(decodedToken)) {
    response.success = false;
    response.message = { ...ERR_AUTHORIZATION };
    return response;
  }
}

```

In the above code, we can notice that there is an implementation of proper permission check where *modelData* consists of properties (intersected with model table and user's permission table) such as *creatable*, *canCreate* which indicates whether or not a model can practically be created, and whether or not the request's user has such permission, respectively.

We should keep in mind that design phase is the most vital and time consuming phase where we put all our efforts, research, and practical implications. We should think for possibly all the scenarios and implement proper measures as well to avoid any unexpected behavior from the code/program that may lead someone take advantage of it.